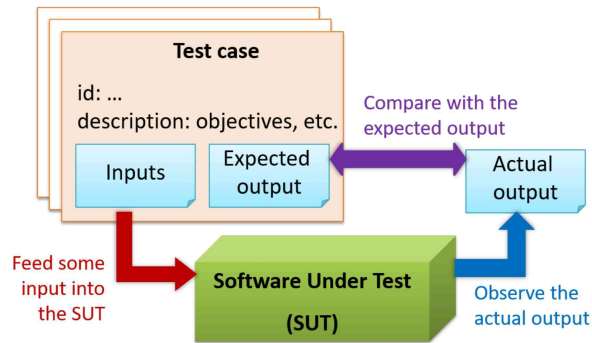


Testing

Introduction

What ⚡⚡

 **Testing:** Operating a system or component under specified conditions, observing or recording the results, and making an evaluation of some aspect of the system or component. -- source: IEEE



When testing, you execute a set of test cases. A test case specifies how to perform a test. At a minimum, it specifies the input to the *software under test (SUT)* and the expected behavior.

 Example: A minimal test case for testing a browser:

- **Input** – Start the browser using a blank page (vertical scrollbar disabled). Then, load `longfile.html` located in the `test data` folder.
- **Expected behavior** – The scrollbar should be automatically enabled upon loading `longfile.html`.

Test cases can be determined based on the specification, reviewing similar existing systems, or comparing to the past behavior of the SUT.

For each test case you should do the following:

1. Feed the input to the SUT
2. Observe the actual output
3. Compare actual output with the expected output

A test case *failure* is a mismatch between the expected behavior and the actual behavior. A failure indicates a potential *defect* (or a *bug*) -- we say 'potential' because the error could be in the test case itself.

 Example: In the browser example above, a test case failure is implied if the scrollbar remains disabled after loading `longfile.html`. The defect/bug causing that failure could be an uninitialized variable.

Testability ⚡⚡⚡

Testability is an indication of how easy it is to test an SUT. As testability depends a lot on the design and implementation, you should try to increase the testability when you design and implement software. The higher the testability, the easier it is to achieve better quality software.

Testing types

Regression testing

What ⚡⚡

When you modify a system, the modification may result in some unintended and undesirable effects on the system. Such an effect is called a **regression**.

Regression testing is the re-testing of the software to detect regressions. The typical way to detect regressions is retesting all related components, even if they had been tested before.

Regression testing is more effective when it is done frequently, after each small change. However, doing so can be prohibitively expensive if testing is done manually. Hence, **regression testing is more practical when it is automated**.

Developer testing

What ⚡⚡

Developer testing is the testing done by the developers themselves as opposed to dedicated testers or end-users.

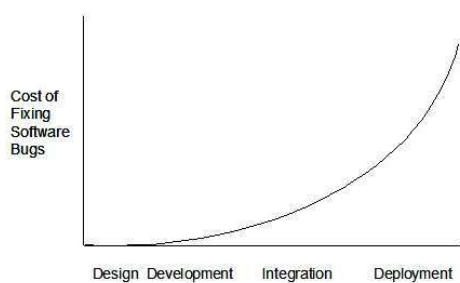
Why ⚡⚡⚡

Delaying testing until the full product is complete has a number of disadvantages:

- **Locating the cause of a test case failure is difficult due to the larger search space;** in a large system, the search space could be millions of lines of code, written by hundreds of developers! The failure may also be due to multiple inter-related bugs.
- **Fixing a bug found during such testing could result in major rework,** especially if the bug originated from the design or during requirements specification i.e. a faulty design or faulty requirements.
- **One bug might 'hide' other bugs,** which could emerge only after the first bug is fixed.
- **The delivery may have to be delayed** if too many bugs are found during testing.

Therefore, it is better to do early testing, as hinted by the popular rule of thumb given below, also illustrated by the graph below it.

The earlier a bug is found, the easier and cheaper to have it fixed.



Such early testing software is usually, and often by necessity, done by the developers themselves i.e., developer testing.

Unit testing

What ⚡

 **Unit testing:** testing individual units (methods, classes, subsystems, ...) to ensure each piece works correctly.

In OOP code, it is common to write one or more unit tests for each public method of a class.

 Here are the code skeletons for a `Foo` class containing two methods and a `FooTest` class that contains unit tests for those two methods.

```
1 class Foo {
2     String read() {
3         // ...
4     }
5 }
```

```

6     void write(String input) {
7         // ...
8     }
9
10 }

```

```

1 class FooTest {
2
3     @Test
4     void read() {
5         // a unit test for Foo#read() method
6     }
7
8     @Test
9     void write_emptyInput_exceptionThrown() {
10        // a unit tests for Foo#write(String) method
11    }
12
13    @Test
14    void write_normalInput_writtenCorrectly() {
15        // another unit tests for Foo#write(String) method
16    }
17 }

```

Stubs ⚡⚡⚡

A proper unit test requires the **unit to be tested in isolation** so that bugs in the dependencies cannot influence the test i.e. bugs outside of the unit should not affect the unit tests.

📦 If a **Logic** class depends on a **Storage** class, unit testing the **Logic** class requires isolating the **Logic** class from the **Storage** class.

Stubs can isolate the SUT from its dependencies.

📖 **Stub:** A stub has the same interface as the component it replaces, but its implementation is so simple that it is unlikely to have any bugs. It mimics the responses of the component, but only for a limited set of predetermined inputs. That is, it does not know how to respond to any other inputs. Typically, these mimicked responses are hard-coded in the stub rather than computed or retrieved from elsewhere, e.g. from a database.

📦 Consider the code below:

```

1 class Logic {
2     Storage s;
3
4     Logic(Storage s) {
5         this.s = s;
6     }
7
8     String getName(int index) {
9         return "Name: " + s.getName(index);
10    }
11 }

```

```

1 interface Storage {
2     String getName(int index);
3 }

```

```

1 class DatabaseStorage implements Storage {
2
3     @Override

```

```

4     public String getName(int index) {
5         return readValueFromDatabase(index);
6     }
7
8     private String readValueFromDatabase(int index) {
9         // retrieve name from the database
10    }
11 }

```

Normally, you would use the `Logic` class as follows (note how the `Logic` object depends on a `DatabaseStorage` object to perform the `getName()` operation):

```

1 Logic logic = new Logic(new DatabaseStorage());
2 String name = logic.getName(23);

```

You can test it like this:

```

1 @Test
2 void getName() {
3     Logic logic = new Logic(new DatabaseStorage());
4     assertEquals("Name: John", logic.getName(5));
5 }

```

However, this `logic` object being tested is making use of a `DatabaseStorage` object which means a bug in the `DatabaseStorage` class can affect the test. Therefore, this test is not testing `Logic` in isolation from its dependencies and hence it is not a pure unit test.

Here is a stub class you can use in place of `DatabaseStorage` :

```

1 class StorageStub implements Storage {
2
3     @Override
4     public String getName(int index) {
5         if (index == 5) {
6             return "Adam";
7         } else {
8             throw new UnsupportedOperationException();
9         }
10    }
11 }

```

Note how the `StorageStub` has the same interface as `DatabaseStorage` , but is so simple that it is unlikely to contain bugs, and is pre-configured to respond with a hard-coded response, presumably, the correct response `DatabaseStorage` is expected to return for the given test input.

Here is how you can use the stub to write a unit test. This test is not affected by any bugs in the `DatabaseStorage` class and hence is a pure unit test.

```

1 @Test
2 void getName() {
3     Logic logic = new Logic(new StorageStub());
4     assertEquals("Name: Adam", logic.getName(5));
5 }

```

In addition to Stubs, there are other type of replacements you can use during testing, e.g. *Mocks*, *Fakes*, *Dummies*, *Spies*.

Integration testing

What ⚡⚡

Integration testing : testing whether different parts of the software work together (i.e. integrates) as expected. Integration tests aim to discover bugs in the 'glue code' related to how components interact with each other. These bugs are often the result of misunderstanding what the parts are supposed to do vs what the parts are actually doing.

📦 Suppose a class `Car` uses classes `Engine` and `Wheel`. If the `Car` class assumed a `Wheel` can support a speed of up to 200 mph but the actual `Wheel` can only support a speed of up to 150 mph, it is the integration test that is supposed to uncover this discrepancy.

How ⚡⚡

Integration testing is not simply a case of repeating the unit test cases using the actual dependencies (instead of the stubs used in unit testing). Instead, integration tests are additional test cases that focus on the interactions between the parts.

📦 Suppose a class `Car` uses classes `Engine` and `Wheel`. Here is how you would go about doing pure integration tests:

- First, unit test `Engine` and `Wheel`.
- Next, unit test `Car` in isolation of `Engine` and `Wheel`, using stubs for `Engine` and `Wheel`.
- After that, do an integration test for `Car` by using it together with the `Engine` and `Wheel` classes to ensure that `Car` integrates properly with the `Engine` and the `Wheel`.

In practice, developers often use a hybrid of unit+integration tests to minimize the need for stubs.

📦 Here's how a hybrid unit+integration approach could be applied to the same example used above:

- First, unit test `Engine` and `Wheel`.
- ~~Next, unit test `Car` in isolation of `Engine` and `Wheel`, using stubs for `Engine` and `Wheel`.~~
- After that, do an integration test for `Car` by using it together with the `Engine` and `Wheel` classes to ensure that `Car` integrates properly with the `Engine` and the `Wheel`. This step should include test cases that are meant to unit test `Car` (i.e. test cases used in the step (b) of the example above) as well as test cases that are meant to test the integration of `Car` with `Wheel` and `Engine` (i.e. pure integration test cases used of the step (c) in the example above).

💡 Note that you no longer need stubs for `Engine` and `Wheel`. The downside is that `Car` is never tested in isolation of its dependencies. Given that its dependencies are already unit tested, the risk of bugs in `Engine` and `Wheel` affecting the testing of `Car` can be considered minimal.

System testing

What ⚡

📖 **System testing: take the whole system and test it against the system specification.**

System testing is typically done by a testing team (also called a QA team).

System test cases are based on the specified external behavior of the system. Sometimes, system tests go beyond the bounds defined in the specification. This is useful when testing that the system fails 'gracefully' when pushed beyond its limits.

📦 Suppose the SUT is a browser that is supposedly capable of handling web pages containing up to 5000 characters. Given below is a test case to test if the SUT fails gracefully if pushed beyond its limits.

Test case: load a web page that is too big
* Input: loads a web page containing more than 5000 characters.
* Expected behavior: aborts the loading of the page and shows a meaningful error message.

This test case would fail if the browser attempted to load the large file anyway and crashed.

System testing includes testing against non-functional requirements too. Here are some examples:

- *Performance testing* – to ensure the system responds quickly.

- *Load testing* (also called *stress testing* or *scalability testing*) – to ensure the system can work under heavy load.
- *Security testing* – to test how secure the system is.
- *Compatibility testing, interoperability testing* – to check whether the system can work with other systems.
- *Usability testing* – to test how easy it is to use the system.
- *Portability testing* – to test whether the system works on different platforms.

Alpha and beta testing

What ⚡⚡⚡

Alpha testing is performed by the users, under controlled conditions set by the software development team.

Beta testing is performed by a selected subset of target users of the system in their natural work setting.

An *open beta release* is the release of not-yet-production-quality-but-almost-there software to the general population. For example, Google's Gmail was in 'beta' for many years before the label was finally removed.

Dogfooding

What ⚡⚡⚡

Dogfooding is when creators use their own product in order to experience how end users experience the product. The term is supposedly derived from the phrase "eating our own dogfood". Dogfooding is different from regular testing in that you *become* an end user, rather than *pretend to be* an end user.

For example, suppose a company produces an email client software. Then, getting some of the employees to use that software for their day-to-day emailing would be dogfooding. Such longer-term, consistent, and authentic use of the software can point to areas of improvement that regular testing (which is often short-term and 'simulated') might not encounter.

Note that dogfooding to be useful, observations need to be deliberately collected and processed i.e., just using the product itself is not enough.

Exploratory versus scripted testing

What ⚡⚡⚡

Here are two alternative approaches to testing a software: *Scripted testing* and *Exploratory testing*.

1. **Scripted testing:** First write a set of test cases based on the expected behavior of the SUT, and then perform testing based on that set of test cases.
2. **Exploratory testing:** Devise test cases on-the-fly, creating new test cases based on the results of the past test cases.

Exploratory testing is 'the simultaneous learning, test design, and test execution' [source: [bach-et-explained](#)] whereby the nature of the follow-up test case is decided based on the behavior of the previous test cases. In other words, running the system and trying out various operations. It is called *exploratory testing* because testing is driven by observations during testing. Exploratory testing usually starts with areas identified as error-prone, based on the tester's past experience with similar systems. One tends to conduct more tests for those operations where more faults are found.

 Here is an example thought process behind a segment of an exploratory testing session:

"Hmm... looks like feature x is broken. This usually means feature n and k could be broken too; you need to look at them soon. But before that, you should give a good test run to feature y because users can still use the product if feature y works, even if x doesn't work. Now, if feature y doesn't work 100%, you have a major problem and this has to be made known to the development team sooner rather than later..."

 **Exploratory testing is also known as *reactive testing*, *error guessing technique*, *attack-based testing*, and *bug hunting*.**

When ⚡⚡⚡

Which approach is better – **scripted or exploratory**? **A mix is better.**

The success of exploratory testing depends on the tester's prior experience and intuition. Exploratory testing should be done by experienced testers, using a clear strategy/plan/framework. Ad-hoc exploratory testing by unskilled or inexperienced testers without a clear strategy is not recommended for real-world non-trivial systems. While **exploratory testing may allow us to detect some problems in a relatively short time, it is not prudent to use exploratory testing as the sole means of testing a critical system.**

Scripted testing is more systematic, and hence, likely to discover more bugs given sufficient time, while exploratory testing would aid in quick error discovery, especially if the tester has a lot of experience in testing similar systems.

In some contexts, you will achieve your testing mission better through a more scripted approach; in other contexts, your mission will benefit more from the ability to create and improve tests as you execute them. I find that most situations benefit from a mix of scripted and exploratory approaches. --[source: bach-et-explained]

Acceptance testing

What ⚡⚡

 **Acceptance testing (aka User Acceptance Testing (UAT): test the system to ensure it meets the user requirements.**

Acceptance tests give an assurance to the customer that the system does what it is intended to do. Acceptance test cases are often defined at the beginning of the project, usually based on the use case specification. Successful completion of UAT is often a prerequisite to the project sign-off.

Acceptance vs System Testing ⚡⚡⚡

Acceptance testing comes after system testing. Similar to system testing, acceptance testing involves testing the whole system.

Some differences between system testing and acceptance testing:

System Testing	Acceptance Testing
Done against the system specification	Done against the requirements specification
Done by testers of the project team	Done by a team that represents the customer
Done on the development environment or a test bed	Done on the deployment site or on a close simulation of the deployment site
Both negative and positive test cases	More focus on positive test cases

Note: *negative* test cases: cases where the SUT is not expected to work normally e.g. incorrect inputs; *positive* test cases: cases where the SUT is expected to work normally

Requirement specification versus system specification

The requirement specification need not be the same as the system specification. Some example differences:

Requirements specification	System specification
limited to how the system behaves in normal working conditions	can also include details on how it will fail gracefully when pushed beyond limits, how to recover, etc. specification
written in terms of problems that need to be solved (e.g. provide a method to locate an email quickly)	written in terms of how the system solves those problems (e.g. explain the email search feature)
specifies the interface available for intended end-users	could contain additional APIs not available for end-users (for the use of developers/testers)

However, in many cases one document serves as both a requirement specification and a system specification.

Passing system tests does not necessarily mean passing acceptance testing. Some examples:

- The system might work on the testbed environments but might not work the same way in the deployment environment, due to subtle differences between the two environments.
- The system might conform to the system specification but could fail to solve the problem it was supposed to solve for the user, due to flaws in the system design.

Test automation

What ⚡⚡



An automated test case can be run programmatically and the result of the test case (pass or fail) is determined programmatically. Compared to manual testing, automated testing reduces the effort required to run tests repeatedly and increases precision of testing (because manual testing is susceptible to human errors).

Automated Testing of CLI Apps ⚡⚡

A simple way to semi-automate testing of a CLI (Command Line Interface) app is by using input/output re-direction. Here are the high-level steps:

- First, you feed the app with a sequence of test inputs that is stored in a file while redirecting the output to another file.
- Next, you compare the actual output file with another file containing the expected output.

Let's assume you are testing a CLI app called `AddressBook`. Here are the detailed steps:

1. Store the test input in the text file `input.txt`.
2. Store the output you expect from the SUT in another text file `expected.txt`.
3. Run the program as given below, which will redirect the text in `input.txt` as the input to `AddressBook` and similarly, will redirect the output of `AddressBook` to a text file `output.txt`. Note that this does not require any changes in `AddressBook` code.

```
java AddressBook < input.txt > output.txt
```

- 💡 The way to run a CLI program differs based on the language.
e.g., In Python, assuming the code is in `AddressBook.py` file, use the command
`python AddressBook.py < input.txt > output.txt`
 - 💡 If you are using Windows, use a normal MS-DOS terminal (i.e., `cmd.exe`) to run the app, not a PowerShell window.
4. Next, you compare `output.txt` with the `expected.txt`. This can be done using a utility such as Windows' `FC` (i.e. File Compare) command, Unix's `diff` command, or a GUI tool such as *WinMerge*.

```
FC output.txt expected.txt
```

Note that the above technique is only suitable when testing CLI apps, and only if the exact output can be predetermined. If the output varies from one run to the other (e.g. it contains a time stamp), this technique will not work. In those cases, you need more sophisticated ways of automating tests.

Test Automation Using Test Drivers ⚡⚡⚡

A test driver is the code that 'drives' the SUT for the purpose of testing i.e. invoking the SUT with test inputs and verifying if the behavior is as expected.

📦 `PayrollTest` 'drives' the `Payroll` class by sending it test inputs and verifies if the output is as expected.

```
1 public class PayrollTest {
2     public static void main(String[] args) throws Exception {
3
4         // test setup
5         Payroll p = new Payroll();
6
7         // test case 1
8         p.setEmployees(new String[]{"E001", "E002"});
9         // automatically verify the response
10        if (p.totalSalary() != 6400) {
11            throw new Error("case 1 failed ");
12        }
13
14        // test case 2
15        p.setEmployees(new String[]{"E001"});
16        if (p.totalSalary() != 2300) {
17            throw new Error("case 2 failed ");
18        }
19
20        // more tests...
21
22        System.out.println("All tests passed");
23    }
24 }
```

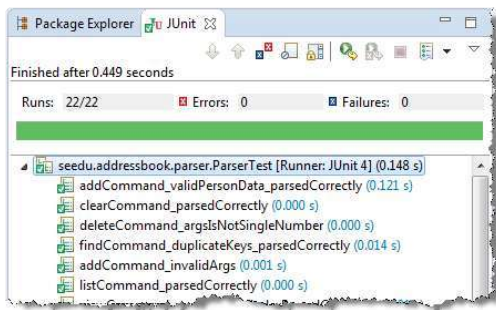
Test Automation Tools ⚡⚡⚡

JUnit is a tool for automated testing of Java programs. Similar tools are available for other languages and for automating different types of testing.

📦 This is an automated test for a `Payroll` class, written using JUnit libraries.

```
16 // other test methods
17
18 @Test
19 public void testTotalSalary() {
20     Payroll p = new Payroll();
21
22     // test case 1
23     p.setEmployees(new String[]{"E001", "E002"});
24     assertEquals(6400, p.totalSalary());
25
26     // test case 2
27     p.setEmployees(new String[]{"E001"});
28     assertEquals(2300, p.totalSalary());
29
30     // more tests...
31 }
```

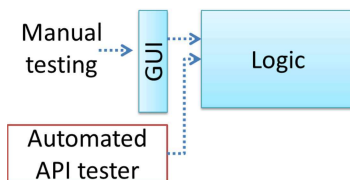
Most modern IDEs have integrated support for testing tools. The figure below shows the JUnit output when running some JUnit tests using the Eclipse IDE.



Automated Testing of GUIs ⚡⚡

If a software product has a GUI (Graphical User Interface) component, all product-level testing (i.e. the types of testing mentioned above) need to be done using the GUI. However, **testing the GUI is much harder than testing the CLI (Command Line Interface) or API**, for the following reasons:

- Most GUIs can support a large number of different operations, many of which can be performed in any arbitrary order.
- GUI operations are more difficult to automate than API testing. Reliably automating GUI operations and automatically verifying whether the GUI behaves as expected is harder than calling an operation and comparing its return value with an expected value. Therefore, automated regression testing of GUIs is rather difficult.
- The appearance of a GUI (and sometimes even behavior) can be different across platforms and even environments. For example, a GUI can behave differently based on whether it is minimized or maximized, in focus or out of focus, and in a high resolution display or a low resolution display.



Moving as much logic as possible out of the GUI can make GUI testing easier. That way, you can bypass the GUI to test the rest of the system using automated API testing. While this still requires the GUI to be tested, the number of such test cases can be reduced as most of the system will have been tested using automated API testing.

There are testing tools that can automate GUI testing.

Some tools used for automated GUI testing:

- **TestFX** can do automated testing of JavaFX GUIs
- **Visual Studio** supports the 'record replay' type of GUI test automation.
- **Selenium** can be used to automate testing of web application UIs

Test coverage

What ⚡⚡

Test coverage is a metric used to measure the extent to which testing exercises the code i.e., how much of the code is 'covered' by the tests.

Here are some examples of different coverage criteria:

- **Function/method coverage** : based on functions executed e.g., testing executed 90 out of 100 functions.
- **Statement coverage** : based on the number of lines of code executed e.g., testing executed 23k out of 25k LOC.
- **Decision/branch coverage** : based on the decision points exercised e.g., an `if` statement evaluated to both `true` and `false` with separate test cases during testing is considered 'covered'.
- **Condition coverage** : based on the boolean sub-expressions, each evaluated to both true and false with different test cases. Condition coverage is not the same as the decision coverage.

📦 `if(x > 2 && x < 44)` is considered one decision point but two conditions.

For 100% branch or decision coverage, two test cases are required:

- `(x > 2 && x < 44) == true` : [e.g. `x == 4`]
- `(x > 2 && x < 44) == false` : [e.g. `x == 100`]

For 100% condition coverage, three test cases are required:

- `(x > 2) == true` , `(x < 44) == true` : [e.g. `x == 4`] [see note 1]
- `(x < 44) == false` : [e.g. `x == 100`]
- `(x > 2) == false` : [e.g. `x == 0`]

Note 1: A case where both conditions are `true` is needed because most execution environments use a *short circuiting* behavior for compound boolean expressions e.g., given an expression `c1 && c2`, `c2` will not be evaluated if `c1` is `false` (as the final result is going to be `false` anyway).

- **Path coverage** measures coverage in terms of possible paths through a given part of the code executed. 100% path coverage means all possible paths have been executed. A commonly used notation for path analysis is called the *Control Flow Graph (CFG)*.

📦 Consider the following Java method.

```
1 void findRate(int input) {
2     if (input == 0) {
3         return 0;
4     }
5     cap = 100/input;
6     if (cap < 0) {
7         return -1;
8     } else {
9         return cap;
10    }
11 }
```

It has 3 paths, as follows:

1. enter -> 2 -> 3 -> exit (can be triggered by input 0)
2. enter -> 2 -> 5 -> 6 -> 7 -> exit (can be triggered by input -5)
3. enter -> 2 -> 5 -> 6 -> 9 -> exit (can be triggered by input 8)

So, to achieve 100% path coverage, we need at least 3 test cases (e.g., 0, -5, 8).

📦 A loop can increase the path count greatly.

```
1 void sayHello(List<String> names) {
2     for (String n : names) {
3         System.out.println(n);
4     }
5 }
```

The number of paths through this method is very large, as each possible length of `names` produces a unique path.

1. enter -> 2 -> exit (if `names` is empty)
2. enter -> 2 -> 3 -> exit (if `names` has one entry)
3. enter -> 2 -> 3 -> 2 -> 3 -> exit (if `names` has two entries) 1 ...

So, achieving 100% path coverage of this method will be extremely difficult.

- **Entry/exit coverage** measures coverage in terms of possible *calls to* and *exits* from the operations in the SUT.

Entry points refer to all places from which the method is called from the rest of the code i.e., all places where the control is handed over to the method in concern.

Exit points refer to points at which the control is returned to the caller e.g., return statements, throwing of exceptions.

How ⚡⚡⚡

Measuring coverage is often done using coverage analysis tools. Most IDEs have inbuilt support for measuring test coverage, or at least have plugins that can measure test coverage.

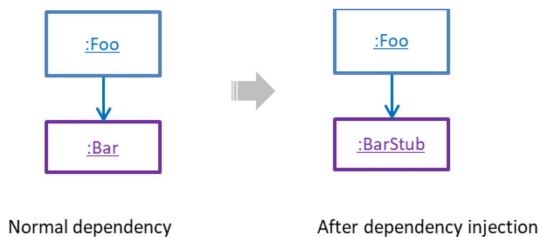
Coverage analysis can be useful in improving the quality of testing e.g., if a set of test cases does not achieve 100% branch coverage, more test cases can be added to cover missed branches.

Dependency injection

What ⚡⚡⚡

Dependency injection is the process of 'injecting' objects to replace current dependencies with a different object. This is often used to inject stubs to isolate the SUT from its dependencies so that it can be tested in isolation.

📦 A `Foo` object normally depends on a `Bar` object, but you can inject a `BarStub` object so that the `Foo` object no longer depends on a `Bar` object. Now you can test the `Foo` object in isolation from the `Bar` object.



TDD

What ⚡⚡⚡

Test-Driven Development(TDD) advocates writing the tests before writing the SUT, while evolving functionality and tests in small increments. In TDD you first define the precise behavior of the SUT using test code, and then update the SUT to match the specified behavior. While TDD has its fair share of detractors, there are many who consider it a good way to reduce defects. One big advantage of TDD is that it guarantees the code is testable.